Portability

The submission must run on a Linux x86-64 machine other than the one it was built on, so the suite is checked in clean containers rather than only on the development host. This page records what was checked, how, and the result. It covers requirement R5 (reproducible transfer) at the level of the environment.

Target

- Linux x86-64.
- Python 3.12 or newer.
- No system services and no network at run time; every command runs offline.

Checks

Environment	Python	Command	Result
Ubuntu 22.04 (glibc 2.35)	3.12	`scripts/test_in_docker.sh ubuntu:22.04 3.12`	183 passed, 1 skipped
Ubuntu 24.04 (glibc 2.39)	3.13	`scripts/test_in_docker.sh ubuntu:24.04 3.13`	183 passed, 1 skipped
Debian 13 development host	3.13	`pytest tests/ -q`	183 passed, 1 skipped
Debian 13, participant machine	3.12	`pytest tests/ -q`	suite green (see "Submission revision")
Fresh clone, new venv	3.13	`pip install -e ".[dev]"` + README/USAGE commands	all run

Those 183-test rows are from the capture, project and report revision. The suite has since grown with the protocol and window modules; the authoritative counts for the submitted revision are in "Submission revision" below.

Submission revision (794 tests)

The submission revision is checked from a clean unpack, so the archived tree and not the working copy is what runs. The counts below are those of the frozen `v1.0.0` archive; the working `main` has since grown to 957 tests, which is expected because it carries work made after the tag. The skip below is the archive's only known difference from a checkout: a zip drops the file mode, so the one test that reads a script's executable bit is skipped rather than failed.

Source	Python	Suite	Lint
Git checkout at this head	3.13.5	794 passed, 0 skipped	All checks passed
`submission_*.zip` from release v1.0.0, clean unpack	3.13.5	793 passed, 1 skipped	All checks passed

No other test skipped on either: the corpus, the reference export and the `examples/` rules are inside the archive. The archive rows were measured on the published `v1.0.0` asset (`submission_20261012_1944.zip`, 517 entries), which is the exact file a reviewer downloads; the checkout row is the same suite at `main`.

Docker

The `Dockerfile` and `scripts/test_in_docker.sh` are provided, but the Docker daemon is not reachable in the environment this revision was checked in, so the container path is prepared and not re-run here. The clean-unpack rows above use a real 3.12 interpreter on the host as the equivalent check. On a host with Docker, `scripts/test_in_docker.sh` runs the same suite and the starter lint in a throwaway Ubuntu image.

What each check does

`scripts/test_in_docker.sh` copies the working tree into an image, installs the pinned dependencies into a virtual environment, and runs the suite. It takes an image and a Python version, so the same script covers both ends of the supported range:

```
scripts/test_in_docker.sh                # ubuntu:22.04, python 3.12
scripts/test_in_docker.sh ubuntu:24.04 3.13
```

The distribution Python on Ubuntu 22.04 is 3.10, so the script adds the deadsnakes PPA when the requested interpreter is not the distribution one. The build steps are chained into one `RUN` line so the Dockerfile stays a single instruction. The script uses the Docker daemon directly when the current user can reach the socket and falls back to `sudo -n` when it cannot.

Dependencies in a clean environment

- `PySide6==6.8.1` installs from the `manylinux_2_28` wheel, which needs glibc 2.28 or newer. Ubuntu 22.04 and 24.04 both satisfy this. No build tools, no compiler, and no system Qt are required; the wheel carries the Qt libraries.
- `dpkt==1.9.8`, `PyYAML==6.0.2`, and the pinned development tools install without errors.
- No install warns or fails on either Python version.

Submission archive check

The submission archive is verified on its own, from a clean unpack rather than from the repository, so a forgotten file cannot hide behind the working tree.

Field	Value
Date	2026-10-11 13:16 UTC
Host	Debian GNU/Linux 13 (trixie), kernel 6.8.0-1055-gke, x86_64, 4 CPUs, 15 GiB RAM
Python	3.13.5 (the archive targets 3.12 or newer)
Archive	`submission_20261012_1944.zip` (the published `v1.0.0` asset), 517 entries
Install time	a fresh venv with `pip install -e ".[dev]"`
Suite	793 passed, 1 skipped (the executable-bit test; see above)
Static check	`ruff check . --select "E4,E7,E9,F" --ignore E501`: All checks passed
Pipeline	`src.project.cli pipeline` with the corpus rule: 3 sessions, `REPORT.md` and `REPORT.html` written
Window	opened headless against the reference export and the corpus rule

The first build omitted `examples/`, which holds the corpus rules. That made 39 corpus and window tests skip and left the walkthrough rule out of the archive. `examples/` is now included and a reference rule is a required entry, so the build fails rather than shipping without it.

Findings

No environment portability defect was found; the one archive-content defect is the missing `examples/` directory recorded above. Two notes are worth keeping:

- `git` is not present in a minimal container. The submission builder reads the author from `git config` when rendering its manifest and treats a missing `git` as an empty identity rather than failing, so the builder also runs in a stripped environment.
- The suite must not depend on a working tree. It does not: the corpus and the reference exports are committed, and `scripts/generate_corpus` rebuilds them deterministically when needed.

Reproducing without Docker

Where Docker is unavailable, a clean virtual environment on any Linux x86-64 host reproduces the check:

```
git clone https://github.com/ForgeLabHub/ForgeMadrigalProtocolLab.git
cd madrigal-protocol-lab
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -e ".[dev]"
pytest tests/ -q
```

Expected: the suite passes. Then run the walkthrough in `docs/USAGE.md`.